SG-5: Drowsiness Decision Module

SG-5 is the **decision stage** of the driver drowsiness detection pipeline. It takes the temporal eye and mouth measurements produced by **SG-4** over a sliding window and classifies the driver's state as **NORMAL**, **WARNING** or **DROWSY**, raising an alarm when the driver is drowsy.

This notebook (CV_Project_SG5.ipynb) builds and tests SG-5 on its own, using mock SG-4 inputs, before it is integrated with SG-4 and deployed on the Jetson.

Where SG-5 sits in the pipeline

Camera → ... → SG-4 (temporal features over a window) → SG-5 (decision) → Alarm / UI

SG-4 summarises a time window (10 s in the mock data) into features such as PERCLOS and eye-closure durations. SG-5 turns those features into a single state and an alarm flag.

Input (from SG-4)

Each input is a Python dictionary:

Field	Meaning
frame_id, timestamp_ms	Identify the frame / moment the decision refers to
window_ms	Length of the temporal window (10000 ms)
valid	Whether SG-4 had enough reliable data
window_valid_fraction	Fraction of the window with usable frames (0–1)
perclos	Percentage of eye closure over the window (0–1)
blink_rate_per_min	Blinks per minute
mean_blink_duration_ms	Average blink duration
longest_closure_ms	Longest eye closure in the window
current_closure_ms	Duration of the ongoing closure (0 if eyes are open)
yawn_count	Number of yawns in the window

The decision methods currently use **PERCLOS, longest closure, current closure and yawn count**. Blink rate and mean blink duration are passed through but not yet used.

Output (to downstream / alarm)

Field	Meaning
frame_id, timestamp_ms	Copied from the input
decision_valid	False if the input could not be trusted
state	"NORMAL", "WARNING", "DROWSY" or None if invalid
drowsiness_score	Numeric score (0–1) or None if invalid
alarm_flag	True only when state is DROWSY
alarm_type	"DROWSINESS" when alarmed, otherwise None

Input validation (all methods)

Before any decision is made, every method rejects unreliable data:

- valid == False → decision_valid = False, no state, no alarm
- window_valid_fraction < 0.70 → same as above

This stops the system from raising (or suppressing) alarms based on poor tracking, e.g. when the face is lost for most of the window.

The three decision methods

1. Rule-based (decision_rules.py → make_decision)

Fixed thresholds on individual cues.

DROWSY if any strong cue is present:

- PERCLOS ≥ 0.35
- longest closure ≥ 1500 ms
- current closure ≥ 2000 ms
- yawns ≥ 3

WARNING otherwise, if any moderate cue is present:

- PERCLOS ≥ 0.20
- longest closure ≥ 800 ms
- yawns ≥ 2

NORMAL otherwise.

Scores are fixed per state (0.20 / 0.50 / 0.80), so they indicate the level rather than a continuous confidence.

2. Weighted fusion (weighted_fusion.py → weighted_decision)

Normalises cues to 0–1 and combines them:

perclos_score = perclos (clipped to 0–1)

closure_score = longest_closure / 2000 (clipped to 0–1)

yawn_score = yawns / 3 (clipped to 0–1)

score = 0.50·perclos_score + 0.30·closure_score + 0.20·yawn_score

Score	State
< 0.30	NORMAL
0.30 – < 0.60	WARNING
≥ 0.60	DROWSY

Safety override: current closure ≥ 2000 ms → DROWSY with score 1.0, regardless of the weighted score.

3. Hybrid fusion (hybrid_fusion.py → hybrid_decision)

Combines the strengths of both:

1. **Strong cues** (same as rule-based) → DROWSY directly, score 0.80.
2. Otherwise compute the **weighted score** as above.
3. **WARNING** if any moderate cue is present **or** the weighted score ≥ 0.30.
4. Otherwise **NORMAL**.

A single strong cue can no longer be "averaged away", while several weak cues can still add up to a warning. The continuous weighted score is kept as drowsiness_score for non-drowsy cases.

Test cases

Basic cases (mock_inputs.py)

Case	Description	Expected
Normal	Low PERCLOS, short closures, no yawns	NORMAL
Warning	Moderate PERCLOS and closure, 1 yawn	WARNING
Drowsy	High PERCLOS, long closure, 3 yawns	DROWSY
Long Closure	Normal PERCLOS but 2.2 s ongoing closure	DROWSY
Multiple Moderate Cues	Several moderate cues together	WARNING
Invalid	SG-4 reports invalid data	None

Borderline cases (defined in Cell 8)

These sit just either side of the thresholds to find where methods break:

Case	What it tests	Expected
PERCLOS Below Warning	PERCLOS 0.19	NORMAL
PERCLOS Above Warning	PERCLOS 0.21	WARNING
Closure Below Drowsy	Longest closure 1450 ms	WARNING
Closure Above Drowsy	Longest closure 1550 ms	DROWSY
Yawns Only	2 yawns, normal eyes	WARNING
Low Valid Fraction	Strong cues but only 60% valid data	None
Moderate Combination	Several moderate cues near the boundary	WARNING
High PERCLOS Only	PERCLOS 0.38, nothing else	DROWSY

Results

Test set	Rule-based	Weighted fusion	Hybrid fusion
Basic (6 cases)	6/6 (100%)	6/6 (100%)	—
Borderline (8 cases)	8/8 (100%)	3/8 (37.5%)	8/8 (100%)

What this shows

- **Weighted fusion dilutes strong single cues.** A PERCLOS of 0.38 contributes only 0.19 to the score, so a clearly drowsy driver is classed as NORMAL. The same thing happens with a 1550 ms closure or two yawns on their own.
- **Rule-based** handles every borderline case but produces only three fixed scores and treats cues independently.
- **Hybrid fusion** matches rule-based accuracy while keeping a continuous score that reflects combined evidence, which makes it the preferred candidate for integration.

Limitations

- All test cases are **hand-written mock data**, and the expected labels were set using the same thresholds the rule-based method uses. 100% accuracy here confirms the logic is implemented as designed; it is not evidence of real-world performance.
- Thresholds and weights are initial choices and still need tuning on real SG-4 output (ideally a labelled drowsiness dataset).
- Decisions are made per window with no smoothing, so the state could flicker between frames. Hysteresis or a minimum alarm duration would help.
- Blink rate and mean blink duration are not yet used.

Files

File	Created by	Purpose
mock_inputs.py	Cell 2	Six mock SG-4 input dictionaries
decision_rules.py	Cell 3	Rule-based method
weighted_fusion.py	Cell 4	Weighted-fusion method

File	Created by	Purpose
compare_methods.py	Cell 5	Runs rule-based and weighted methods side by side
hybrid_fusion.py	Cell 10	Hybrid-fusion method
sg5_method_comparison.csv	Cell 12	Borderline comparison table for all three methods

Notebook walkthrough

Cell	What it does
1	Creates and enters /content/module_sg5
2	Writes the mock SG-4 inputs
3	Writes the rule-based method
4	Writes the weighted-fusion method
5–6	Writes and runs the side-by-side comparison script
7	Accuracy of rule-based vs weighted on the basic cases
8	Defines the borderline test cases
9	Accuracy of rule-based vs weighted on borderline cases
10	Writes the hybrid-fusion method
11	Accuracy of all three methods on borderline cases
12	Builds the comparison table and saves it to CSV

How to run

1. Open CV_Project_SG5.ipynb in Google Colab (or Jupyter with Python 3 and pandas).
2. Run all cells in order. Cells 2, 3, 4, 5 and 10 must run before the cells that import from them, and Cell 8 must run before Cells 9, 11 and 12.
3. Outputs print in the notebook; the final table is saved as sg5_method_comparison.csv in /content/module_sg5.

To use a method directly:

```
from hybrid_fusion import hybrid_decision


result = hybrid_decision(sg4_output_dict)

if result["decision_valid"] and result["alarm_flag"]:
    trigger_alarm()
```

Next steps

- Integrate with live SG-4 output and test on the Jetson.
- Tune thresholds and weights on real labelled data.
- Add temporal smoothing / hysteresis to avoid flickering alarms.
- Evaluate with more realistic metrics (per-class precision and recall, false-alarm rate, detection latency).